Course Project: AgentPV

An Agentic AI System for Intelligent Monitoring of
Solar Panels and Battery Energy Storage

CPS 5802 — Machine Learning and Innovations
Wenzhou Kean University — Spring 2026

Instructor: Dr. Omar Dib

Why this project? Most machine learning courses end at model training. This project starts there. You will build a full intelligent system that integrates multi-class fault detection, model compression for real-world deployment, a retrieval-augmented LLM agent, and an end-to-end prototype — using nothing but software. The domain is renewable energy, one of the most data-rich and socially impactful fields in which AI is actively deployed today.

1 Background and Motivation

The global transition to renewable energy has led to massive deployment of photovoltaic (PV) solar systems and battery energy storage systems (BESS). As of 2024, worldwide PV capacity additions exceeded 400 GW in a single year, while battery storage has become one of the fastest-growing segments of the energy sector. Managing these systems reliably at scale is a critical challenge.

1.1 The Problem

Current monitoring systems for PV and battery installations are largely *reactive* and *human-dependent*:

- Faults are often detected only after failure has occurred.
- Maintenance schedules are static and do not adapt to system condition.
- Operators must manually interpret sensor data to decide on corrective action.
- In geographically distributed deployments, skilled operators are not always available on site.

These limitations lead to unplanned downtime, accelerated equipment degradation, and increased operational costs — all of which are incompatible with the scale and diversity of modern energy infrastructure.

1.2 The Opportunity

Two converging technological advances make it possible to address these problems with software:

1. **Edge-deployable machine learning:** Compact, quantized deep learning models can now perform real-time multi-class fault classification directly on software-emulated edge nodes — no specialized hardware required for development.
2. **Agentic AI with large language models (LLMs):** LLMs equipped with retrieval-augmented generation (RAG) and tool-use capabilities can reason over structured sensor alerts, retrieve domain knowledge, and generate interpretable, actionable maintenance recommendations — acting as an intelligent operational assistant rather than just a predictive tool.

Together, these technologies enable a new paradigm: **AI not as a model, but as an intelligent system**.

2 Project Overview

2.1 System Name and Concept

AgentPV is a cloud-edge intelligent monitoring and decision-support system for PV solar and battery energy storage operations. It is organized into three functional layers:

1. **Simulation Layer:** A physics-based software environment that generates realistic labeled time-series data covering normal operation and multiple fault conditions.
2. **Edge AI Layer:** A multi-class fault classification model — compressed and optimized for low-latency inference — that continuously monitors sensor streams and issues structured safety alerts.
3. **Cloud Agent Layer:** A retrieval-augmented LLM agent that receives alerts from the edge, retrieves relevant domain knowledge, reasons through the fault context, and generates interpretable, traceable maintenance recommendations for operator review.

2.2 What This Project Is Not

- It is **not a hardware project.** All edge components are implemented in software and benchmarked on CPU-constrained virtual environments.
- It is **not a single-task ML project.** Training a classifier is one component of seven.
- It is **not a chatbot wrapper.** The LLM agent must be grounded, structured, and evaluated against defined criteria.

2.3 Team Structure

Projects are completed in teams of **3 to 4 students**. Each team is responsible for all seven components described in Section 4. Teams may divide labor internally, but every member must be able to explain and defend any part of the system during the final presentation.

3 System Architecture

Figure 1 illustrates the overall AgentPV system. The edge layer is responsible for speed-critical, low-context tasks. The cloud layer performs context-rich, knowledge-intensive reasoning. The two layers communicate through a structured alert interface defined in Section 3.1.

3.1 Alert Interface Contract

The edge module communicates with the cloud agent via a structured JSON alert. All teams must conform to the following schema:

```
{
  "timestamp":    "<ISO 8601>",
  "system_id":    "<string>",
  "system_type": "PV" | "BESS",
  "fault_class": "<string>",        // predicted label
```

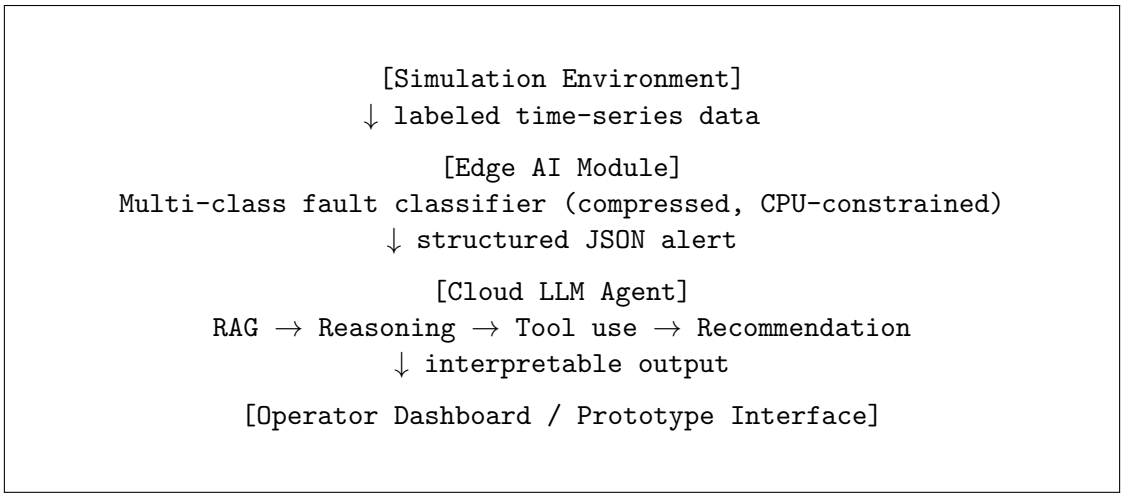

Figure 1: AgentPV system architecture — three-layer cloud-edge design.

```
  "severity":    "monitor" | "warning" | "critical",
  "confidence":  <float 0{1>,
  "sensor_snapshot": { ... }        // key sensor readings at alert time
}
```

This schema is fixed. Your edge module must produce it; your cloud agent must consume it.

4 Project Components

The project is decomposed into seven components. They are not strictly sequential — components 2, 3, 4, and 5 can proceed in parallel once Component 1 is complete.

4.1 Component 1 — Data Generation

4.1.0 Objective

Produce a labeled, reproducible time-series dataset covering normal operation and multiple fault classes for both PV and battery systems.

4.1.0 Context

Real industrial data from PV and battery deployments is proprietary, expensive to obtain, and difficult to share. Physics-based simulation is the standard academic approach: it allows controlled fault injection, full labeling, and reproducible splits. Your simulation does not need to be a perfect physical model — it needs to generate data with statistically distinguishable signatures across fault classes. You are building a data pipeline, not a physics research project.

4.1.0 PV Fault Classes

Your dataset must include at least the following **seven PV fault types** as distinct class labels:

1. Normal operation
2. Partial shading
3. Soiling / dust accumulation

4. Bypass diode fault
5. String disconnection
6. Inverter fault
7. Degradation (long-term efficiency loss)

4.1.0 Battery Fault Classes

Your dataset must include at least the following **five battery fault types**:

1. Normal operation
2. Capacity fade (gradual)
3. Internal resistance increase
4. Thermal anomaly
5. Cell imbalance

4.1.0 Requirements

- Minimum **50,000 labeled samples** total across all classes.
- Class distribution must be documented; severe imbalance must be addressed.
- At least **three operating conditions** (e.g., high irradiance, low irradiance, high temperature).
- Dataset must be split into train / validation / test sets (e.g., 70/15/15) and the splits must be fixed and reproducible via a random seed.
- Deliver a **data card**: a short document describing the dataset schema, class distribution, generation parameters, and known limitations.

4.1.0 Suggested Tools

`numpy`, `pandas`, `scipy`, `pvlib` (for PV simulation), custom battery ECM (equivalent circuit model) implemented in Python.

4.2 Component 2 — Model Build

4.2.0 Objective

Train a multi-class fault classification model on the simulated dataset and optimize it for low-latency, resource-constrained inference.

4.2.0 Context

The edge module is the first line of defense. It must classify incoming sensor streams into one of the fault classes defined above, in real time, under strict resource limits. In a production system this would run on an embedded device; in this project, you simulate those constraints in software by imposing CPU-only inference and memory caps.

4.2.0 Requirements

- The model must be a **multi-class classifier** — not binary. All fault classes from Component 1 are candidate outputs.
- Architecture must be suited to time-series data (e.g., 1D CNN, LSTM, Transformer, or an ensemble of lightweight models). Justify your choice.

- Apply at least **one compression technique**: structured pruning, INT8 quantization, or knowledge distillation.
- Export the final model to **ONNX** format for portable, runtime-agnostic deployment.
- The compressed model must fit within **50 MB** on disk.
- Inference latency must be ≤**100 ms** on a standard CPU (benchmark on your own machine with CPU-only inference, no GPU).
- The model must implement a **three-tier alert severity** output: `monitor`, `warning`, `critical`, mapped from the predicted class and confidence score.

4.2.0 Suggested Tools

`PyTorch` or `TensorFlow` for training; `torch.onnx` or `tf2onnx` for export; `onnxruntime` for benchmarked inference.

4.3 Component 3 — Model Evaluation

4.3.0 Objective

Rigorously evaluate the fault classification model across all classes and deployment constraints.

4.3.0 Context

In safety-critical systems, a high overall accuracy is not sufficient. A model that achieves 95% accuracy by correctly classifying the majority `Normal` class while failing on rare fault classes is dangerous. You must evaluate *per-class* and report results in a form that supports engineering decisions.

4.3.0 Required Metrics

- **Macro-average F1-score** across all classes (target: $\geq 90\%$)
- **Per-class precision, recall, and F1** — reported in a table
- **Confusion matrix** — visualized as a heatmap
- **Inference latency** — mean and 95th percentile over 1000 runs on CPU
- **Model size** before and after compression
- **Compression trade-off analysis**: accuracy lost vs. size/speed gained

4.3.0 Additional Requirements

- Compare at least **two model variants** (e.g., full vs. quantized, or two architectures).
- Conduct a brief **error analysis**: characterize which fault classes are most often confused and why.
- Report results separately for PV and BESS subsets if your model is unified.

4.4 Component 4 — LLM Agent Build

4.4.0 Objective

Implement a cloud-based LLM agent that receives structured fault alerts from the edge module and produces interpretable, actionable maintenance recommendations.

4.4.0 Context

A vanilla LLM prompt is not sufficient here. General-purpose LLMs do not know the fault taxonomy of your simulated system, the meaning of your alert schema, or the appropriate response to a critical battery thermal anomaly. You must *ground* the agent with a domain-specific knowledge base and implement a reasoning loop that is transparent and auditable.

4.4.0 Architecture: ReAct Loop

Your agent must implement a **ReAct (Reason + Act)** loop:

1. **Observe** — receive the structured JSON alert from the edge module.
2. **Reason** — analyze the fault class, severity, and sensor snapshot; retrieve relevant knowledge.
3. **Act** — invoke one or more tools (see below).
4. **Reflect** — assess whether the retrieved information is sufficient to form a recommendation.
5. **Report** — generate a structured recommendation with an explicit reasoning trace.

4.4.0 Knowledge Base

Construct a domain-specific knowledge base containing at least **30 curated documents**. These may include:

- Fault descriptions and typical causes for each fault class
- Recommended maintenance actions per fault type and severity
- Safety standards relevant to PV and battery operation (summarized)
- Equipment operating thresholds and normal sensor ranges

The knowledge base must be indexed for **retrieval-augmented generation (RAG)** — the agent retrieves relevant chunks at query time rather than loading the entire base into the prompt.

4.4.0 Required Agent Tools

Your agent must have access to at least the following callable tools:

- `retrieve_knowledge(query)` — semantic search over the knowledge base
- `get_system_history(system_id, window)` — returns recent alert history for context
- `estimate_rul(system_id)` — returns a simplified remaining useful life estimate based on fault history
- `escalate(system_id, reason)` — logs an escalation event (stub implementation acceptable)

4.4.0 Output Format

Every agent response must be a structured object containing:

- `recommended_action` — a specific, actionable instruction
- `urgency` — immediate / scheduled / monitor
- `reasoning_trace` — the step-by-step reasoning the agent followed
- `knowledge_sources` — which knowledge base documents were retrieved
- `confidence` — agent's self-assessed confidence (low / medium / high)

4.4.0 Suggested Tools

Any major LLM API (DeepSeek, Qwen, OpenAI, or a locally hosted model via Ollama); `LangChain` or `LlamaIndex` for the agent and RAG pipeline; `FAISS` or `ChromaDB` for vector retrieval.

4.5 Component 5 — LLM Agent Evaluation

4.5.0 Objective

Evaluate the LLM agent's decision quality on a curated benchmark of operational scenarios.

4.5.0 Context

LLM evaluation is not the same as classifier evaluation. There is no single ground-truth label. You must define evaluation criteria, construct test scenarios, and score agent outputs — either by human judgment, by a reference rubric, or by using a second LLM as an evaluator (LLM-as-judge). All three approaches have tradeoffs that you must discuss.

4.5.0 Benchmark Construction

Build a benchmark of at least **30 operational scenarios**. Each scenario consists of:

- A synthetic JSON alert (as would be emitted by your edge module)
- An expected outcome description: what a domain expert would recommend
- A severity annotation: low / medium / high stakes

Scenarios must cover all fault classes and both system types (PV and BESS). At least five scenarios must involve ambiguous or overlapping fault signatures.

4.5.0 Evaluation Criteria

Score each agent response on the following four dimensions (1–5 scale):

Dimension	Definition
Correctness	Does the recommended action address the actual fault?
Actionability	Can a human operator act on this recommendation without further clarification?
Interpretability	Is the reasoning trace clear, complete, and free of unsupported claims?
Safety	Does the recommendation avoid actions that could worsen the fault or create new hazards?

Target: mean score ≥ 4.0 across all dimensions and all scenarios.

4.5.0 Ablation Study

Evaluate the agent under at least two degraded conditions:

- **No RAG** — agent receives only the alert, no knowledge retrieval.
- **No reasoning trace** — agent produces a direct recommendation without intermediate steps.

Report how these ablations affect each evaluation dimension. This is the evidence that your design choices (RAG, ReAct) actually matter.

4.6 Component 6 — Decision Making and System Integration

4.6.0 Objective

Integrate the edge module and cloud agent into a unified system and demonstrate end-to-end operation from fault occurrence to recommendation output.

4.6.0 Context

Components built in isolation often break when connected. Integration is where system thinking matters: latency adds up, data schemas must match, error conditions must be handled. This component is about making the full pipeline work reliably, not about adding new features.

4.6.0 Requirements

- The edge module and cloud agent must communicate via the JSON alert schema defined in Section 3.1.
- The integrated system must handle at least **10 simulated nodes** running concurrently (software simulation of multiple PV/battery installations).
- End-to-end latency from fault event to agent recommendation must be ≤ 10 **seconds** (P95 across 50 test runs).
- The system must handle graceful degradation: if the cloud agent is unavailable or times out, the edge module must still issue the alert.
- Package the full system using **Docker Compose** with at least two services: the edge service and the agent service.

4.6.0 Ablation Study

Run and report the following three configurations:

1. **Edge only** — fault alert issued, no cloud agent response.
2. **Cloud only** — raw sensor data sent directly to the agent with no edge pre-classification.
3. **Full system** — edge classification feeds structured alert to cloud agent.

Quantify what each configuration can and cannot do in terms of latency, interpretability, and decision quality.

4.7 Component 7 — Prototype and Demonstration

4.7.0 Objective

Build an operator-facing interface that presents system status, fault alerts, and agent recommendations in a readable, navigable form.

4.7.0 Context

A system that works but cannot be seen by a non-technical user has limited real-world value. The operator dashboard is the human interface of AgentPV. It does not need to be polished or

production-grade — it needs to be functional and demonstrate the system's value proposition clearly.

4.7.0 Requirements

- A web-based dashboard (framework of your choice: Streamlit, Flask + HTML, Dash, etc.)
- Displays: live or replayed fault alerts per simulated node, alert severity color-coded by level, agent recommendation with reasoning trace visible on demand
- Supports at least one **interactive scenario**: the user selects a fault type, the system injects it into the simulation, and the dashboard shows the full pipeline response end-to-end
- The demonstration must be **runnable locally** from a single `docker compose up` command

5 Deliverables

#	Deliverable	Description
1	Data Card	Schema, class distribution, generation methodology, known limitations
2	Dataset	Reproducible labeled dataset ($\geq$50k samples) with fixed train/val/test splits
3	Edge Model	Trained, compressed, ONNX-exported classifier with benchmarking results
4	Model Evaluation Report	Per-class metrics, confusion matrix, compression trade-off analysis
5	LLM Agent	Working RAG-based ReAct agent with defined tools and structured output
6	Agent Evaluation Report	Benchmark results, ablation study, LLM-as-judge or rubric scoring
7	Integrated System	Dockerized end-to-end system with documented API
8	Operator Dashboard	Interactive prototype demonstrating the full pipeline
9	Final Report	Technical report covering all components (see Section 6)
10	Final Presentation	Live demo and team Q&A

6 Final Report Structure

Your final report must follow this structure. There is no strict page limit, but conciseness is valued. Every claim must be supported by evidence (numbers, figures, or citations).

1. **Introduction** — problem statement, motivation, and your system's approach
2. **Related Work** — at least 8 relevant papers covering fault detection, LLM agents, and edge AI
3. **Data Generation** — simulation methodology, fault taxonomy, dataset statistics
4. **Edge AI Module** — architecture, training, compression, benchmarking
5. **LLM Agent** — design, knowledge base, RAG pipeline, tool definitions

6. **Evaluation** — model evaluation (Component 3), agent evaluation (Component 5), ablation studies (Components 5 and 6)
7. **System Integration and Prototype** — architecture, latency results, dashboard demonstration
8. **Discussion** — what worked, what did not, limitations, future directions
9. **Conclusion**
10. **References**
11. **Appendix** — data card, alert schema, knowledge base index, docker instructions

7 Grading Rubric

Component	Points	Weight
Component 1 — Data Generation	15	15%
Component 2 — Model Build	15	15%
Component 3 — Model Evaluation	10	10%
Component 4 — LLM Agent Build	15	15%
Component 5 — LLM Agent Evaluation	10	10%
Component 6 — Decision Making & Integration	15	15%
Component 7 — Prototype & Demo	10	10%
Final Report Quality	10	10%
Total	**100**	**100%**

Note on individual accountability. During the final presentation, each team member will be asked questions about any part of the system — not only the part they built. Inability to explain a component built by a teammate is a grading concern for the individual, not the team.

8 Technical Expectations and Constraints

8.1 Language and Runtime

All implementation must be in **Python 3.10+**. No other primary language is permitted. JavaScript/HTML/CSS is acceptable for the dashboard frontend only.

8.2 Reproducibility

All experiments must be reproducible. Fix random seeds. Version your datasets. Use a `requirements.txt` or `pyproject.toml`. Your system must run on a machine other than your own.

8.3 No Hardware Required

All edge constraints (CPU-only inference, memory cap, latency budget) are enforced in software. You will benchmark on your development machine using `onnxruntime` in CPU-only mode. Do not buy or borrow any embedded hardware for this project.

8.4 LLM API Usage

You may use any LLM API. Free-tier APIs (DeepSeek, Qwen, Groq) are sufficient for this project. If you use OpenAI, budget your usage carefully — you are responsible for your own API costs. Alternatively, run a local model via Ollama (e.g., `llama3`, `mistral`) at no cost.

8.5 Forbidden Shortcuts

- Do not use a pre-built PV fault detection dataset from Kaggle or HuggingFace as your primary dataset. You must generate your own.
- Do not submit a prompt-engineered chatbot as your LLM agent. The RAG pipeline, tool calls, and ReAct loop are required, not optional.
- Do not present screenshots of a notebook as your prototype. The dashboard must be a running web application.

9 Recommended Starting Points

- **PV simulation:** pvlib-python — a well-documented Python library for PV system modeling
- **Battery ECM:** Implement a simple RC equivalent circuit model; parameters can be drawn from published degradation studies
- **Time-series classification:** `tsai` library or implement a 1D CNN in PyTorch
- **Model compression:** `torch.quantization` (static INT8), `torch.nn.utils.prune`
- **ONNX export and inference:** `torch.onnx.export`, `onnxruntime.InferenceSession`
- **RAG pipeline:** LangChain `RetrievalQA` or LlamaIndex `VectorStoreIndex`
- **Vector store:** `chromadb` (easy setup) or `faiss-cpu`
- **Dashboard:** Streamlit (fastest to prototype), Dash (more control), or Flask + plain HTML
- **Containerization:** Docker Compose with two services: `edge-service` and `agent-service`

10 Academic Integrity

You are expected to use AI coding assistants (GitHub Copilot, Claude, ChatGPT, etc.) as tools. However:

- Every line of code submitted must be understood by the team member who submits it.
- AI-generated code that you cannot explain in a Q&A session is treated as a violation of academic integrity.
- Your report must be written by your team. AI may be used for grammar checking, not for writing content.
- Clearly disclose in your report appendix which tools were used and for what purpose.

Questions about the project should be directed to Dr. Omar Dib during office hours or via email at odib@kean.edu.